button's title. So, to not duplicate code, we're only setting the title on the action.

Back in the Rails codebase, open `button_controller.js`. In the body of the callback function, *click* the bridge element, like so:

ch07_12/rails/app/javascript/controllers/bridge/button_controller.js

```
import { BridgeComponent } from "@hotwired/hotwire-native-bridge"

export default class extends BridgeComponent {
  static component = "button"

  connect() {
    super.connect()

    this.send("connect", {}, () => {
»     this.bridgeElement.click()
    })
  }
}
```

`this.bridgeElement` wraps the attached HTML element, the one we added the `controller` data attribute to, in a `BridgeElement` instance. Using this, we can grab the title, check if it's disabled, and even click it. Check out the documentation on Bridge Elements[44] and Data Attributes[45] for more info on what bridge elements can do.

Hop back into Xcode and run the app. Open the sign-in screen and tap "Sign in" at the top right. The sign-in form will submit as if you tapped the HTML submit button. Nice work—your first functional bridge component!

Right now, the native Sign in button pretty much only works as, well, a Sign in button since we hardcoded the title in Swift. This kind of defeats the purpose of bridge components: they're supposed to be dynamic components driven by HTML. Let's change that by setting the title of the button from the server.